University of St.Gallen

Welcome and Introduction

ASSE Reloaded

Simon Mayer, Alessandro Giugno, Luka Bekavac

University of St.Gallen
Master in Computer Science

Advanced Software and Systems Engineering (7,856)

PDF version of 09.11.2026 (the online slides may be newer)



From insight to impact.

Jannis. Personalized Reality.

Sanjiv. Explainable CPS.

Jan. Software.

Jérémy. Interoperable Intelligence.

Andrei. HyperAgents.

Simon. Systems and People.



Damian. Automation and User Trust.

Lukas. Research Entrepreneur.

Jing. Personalized Diet.

Kenan. Gaze.

Danai. HyperAgents Interactions.

Kim. Sustainable Intelligence.

Alessandro. Bio-Inspired Agents



Ganesh. Autonomous Buildings.

Jan. Better Semantic Web.

Luka. Systemic Risks in VLOPs





Clement. Regulation Automation



Has everyone received their briefings?

orinoco web services

Hyperscaler

mingle

Social Media Platform

SÄNTIS
Industrie AG

Legacy Industry Company

kestrel:// foundation

Browser Manufacturer

DFF DIGITAL FREEDOM FRONT

OSS / Privacy Activists

COMMISSION
OF THE CONTINENTAL UNION

Democratic Government / Regulator

Interactions: Our Research

Topic	Selected papers	In ASSE
Hypermedia multi-agent systems	*Yggdrasil: an artifact-based framework for hypermedia multi-agent systems* (2026); *Enabling BDI agents to reason on a dynamic action repertoire in hypermedia environments* (AAMAS 2024)	Lecture 11
Agents and the Web of Things	*Engineering multi-agent systems for the Web of Things* (2026); *A vision for LLM-based interaction assistance for autonomous agents on the Semantic Web* (2025)	Lecture 10, Lecture 11
Semantic interoperability	*Integrating Semantic Web ontologies towards achieving automated engineering of controls in smart buildings* (CASE 2025)	Lecture 10
Decentralized personal data	*Solid and secure multi-party computation for a circular economy* (2026)	Lecture 9, Assignment 9
The Web as a hypermedia system	*Broken links and fading stories: how dominant online platforms subvert the hypermedia foundation of the Web* (WebSci 2026)	Lecture 7b

Google Scholar: Simon Mayer

Excerpts from our Research

Scanzio, Cena, Valenzano: QRScript: Embedding a Programming Language in QR codes to support Decision and Management. IEEE ETFA 2022



QRUco: Interactive QR Codes Through Thermoresponsive Embeddings

Kay Erik Jenß
kayerik.jenss@student.unisg.ch
Institute of Computer Science, University of St.Gallen
St. Gallen, Switzerland

Simon Mayer
simon.mayer@unisg.ch
Institute of Computer Science, University of St.Gallen
St. Gallen, Switzerland

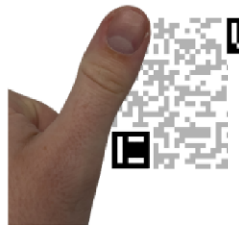
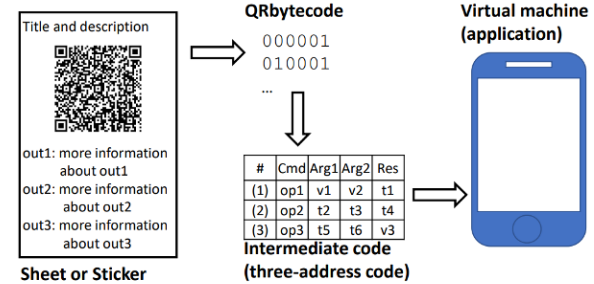


Figure 1: Image of a QRUco ma



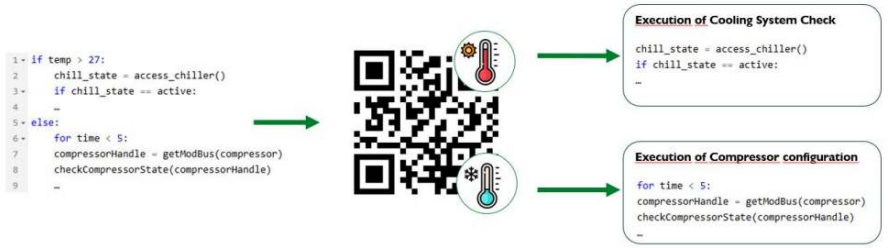


Supernatural Superserious: Reactive Executable Markers

Lukas Eichelberger
lukas.eichelberger@student.unisg.ch
Interactions Research Group,
University of St.Gallen
St. Gallen, Switzerland

Simon Mayer
simon.mayer@unisg.ch
Interactions Research Group,
University of St.Gallen
St. Gallen, Switzerland

Ganesh Ramanathan
ganesh.ramanathan@student.unisg.ch
Interactions Research Group,
University of St.Gallen
St. Gallen, Switzerland



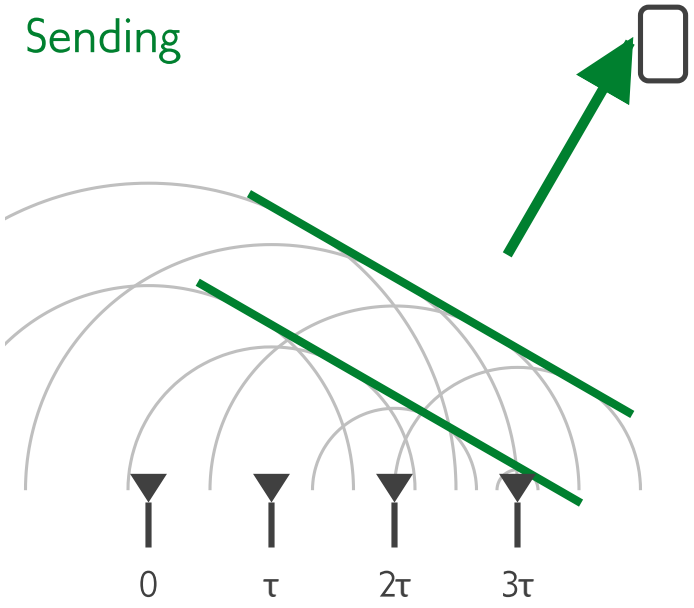


Reactive Executable Markers (REMs) **expose code that changes based on the current ambient temperature.**

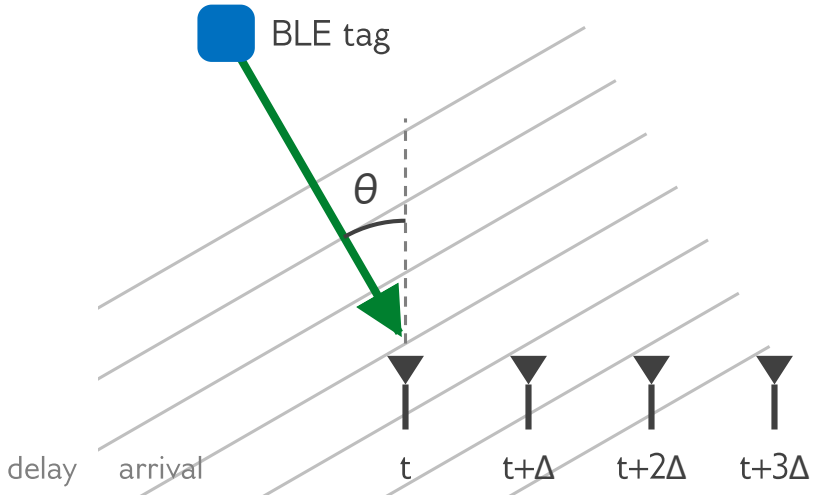
Jenss, Mayer. QRUco: Interactive QR Codes Through Thermoresponsive Embeddings. CHI 2023. Paper & Video · Eichelberger, Mayer, Ramanathan. Supernatural Superserious: Reactive Executable Markers. UbiComp 2025.

What is Reverse Beamforming?

Sending



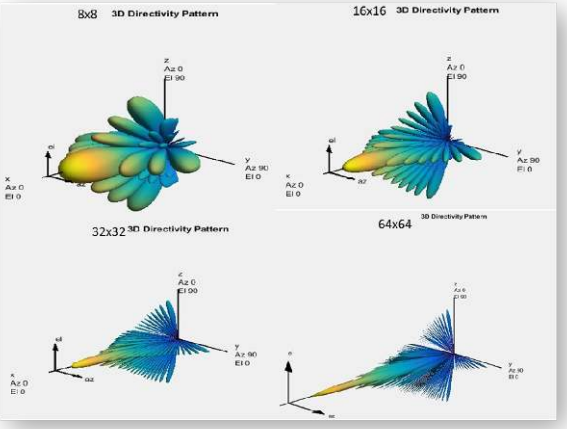
Receiving



Beamforming: several antennas send the same signal, each a little later than its neighbour. The waves add up in one direction (5G, Wi-Fi).

Reverse beamforming: several antennas receive one signal. From the differences in arrival time, we compute where it came from (angle of arrival).

Excerpts from our Research

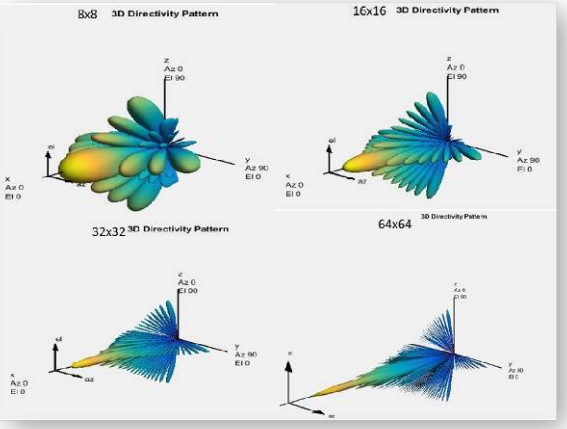


Reverse Beamforming + Visual Object Recognition = ?



Excerpts from our Research





Reverse Beamforming + Visual Object Recognition = ?



Strecker, Akhunov, Carbone, Garcia, Bektas, Gomez, Mayer, Yildirim. *MR Object Identification and Interaction: Fusing Object Situation Information from Heterogeneous Sources*. IMWUT 2023

Simon: Systems and People







PhD in **Computer Science**, ETH Zurich (2010-2014)

Senior Key Expert for Smart and Interoperable Systems, Siemens Corporate Technology, Berkeley (2014-2017)

Research Group Leader "Cognitive Products", Pro²Future (2017-2018)
Postdoctoral Researcher, TU Graz (2017-2018)

Senior Researcher, ETH Zurich (2018-2020)

Professor for Interaction-and Communication-based Systems, University of St.Gallen (2018 →)

Director, Institute of Computer Science, University of St.Gallen (2019 →)

Dean, School of Computer Science, University of St.Gallen (2024-2026)

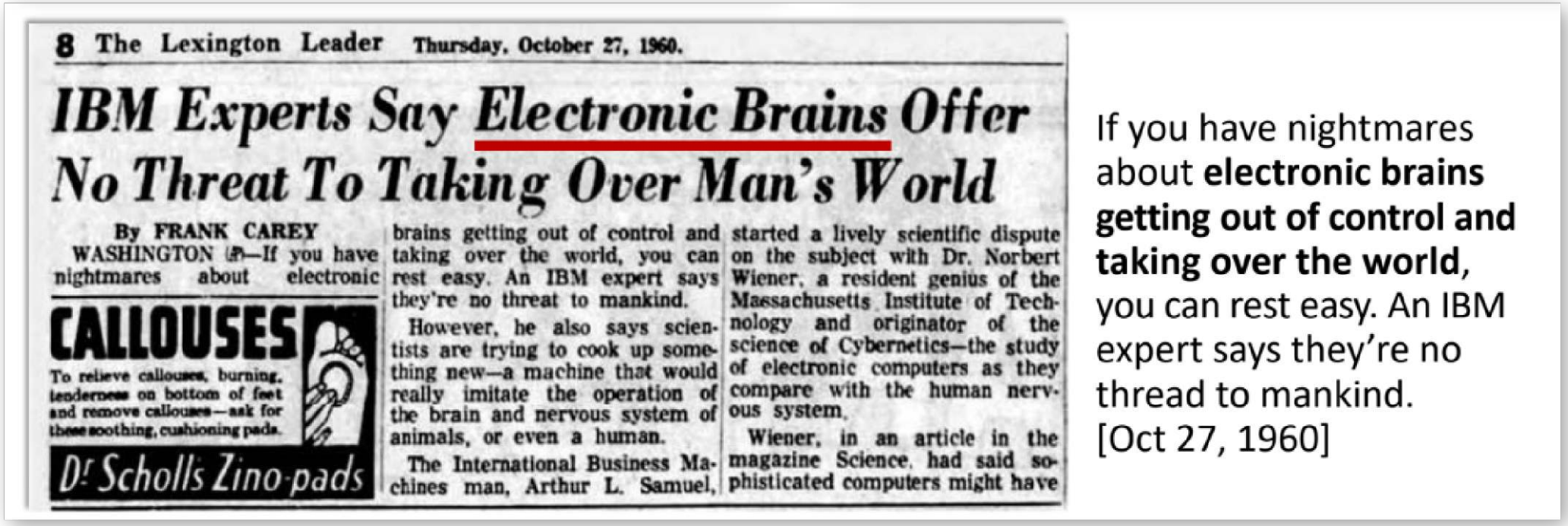
Co-Founder, Future Society Hub (2026 →)

Lecturer, ETH Zurich (2015-2020), MIT (2015), UC Berkeley (2015-2017), TU Graz (2017-2019), University of Zurich (2020-2022), University of Bologna (2023)

3 Children, Backcountry Snowboarding



Will AI Replace Software Engineers?



8 The Lexington Leader Thursday, October 27, 1960.

IBM Experts Say Electronic Brains Offer No Threat To Taking Over Man's World

By FRANK CAREY

WASHINGTON (AP)—If you have nightmares about electronic brains getting out of control and taking over the world, you can rest easy. An IBM expert says they're no threat to mankind.

However, he also says scientists are trying to cook up something new—a machine that would really imitate the operation of the brain and nervous system of animals, or even a human.

The International Business Machines man, Arthur L. Samuel, started a lively scientific dispute on the subject with Dr. Norbert Wiener, a resident genius of the Massachusetts Institute of Technology and originator of the science of Cybernetics—the study of electronic computers as they compare with the human nervous system.

Wiener, in an article in the magazine Science, had said sophisticated computers might have

CALLOUSES
To relieve callouses, burning, tenderness on bottom of feet and remove callouses—ask for these soothing, cushioning pads.
Dr Scholl's Zino-pads

If you have nightmares about **electronic brains getting out of control and taking over the world**, you can rest easy. An IBM expert says they're no thread to mankind. [Oct 27, 1960]

Did computers take our jobs? **What about AI?**

The Lexington Leader, 27 October 1960

80 Years of Programming: **A Quick Run-Through**



Wiring

```
Plug cables
Set switches
```

ENIAC: programming means rewiring.

Days of manual work.

Machine Code

```
LDA A
ADD B
STA C
```

“Planners” design, “coders” translate into instructions by hand.

Compiler

```
C = A + B
```

FORTRAN: the machine translates.

Software Engineering

```
if overdue:
remind()
```

Software becomes an engineering discipline: teams, tests, processes.

Open Source & Web

```
import pandas
```

Reuse instead of rewriting: GitHub, Stack Overflow, libraries.

AI Assistants & Agents

```
“Build me an
app that …”
```

From autocomplete to agents that complete whole tasks on their own.

Humans describe more and more of the What. Machines take over more and more of the How.

The Compiler: Automating Programming in the 1950s

Nobody believed that. I had a running compiler and nobody would touch it. ***They told me computers could only do arithmetic.***

Grace Hopper
on her A-0 compiler (1952)



Photo of Grace Hopper: Wikipedia

1

Translation was automated

The machine took over the work of the “coders”: turning plans into machine instructions by hand.

2

The profession did not disappear

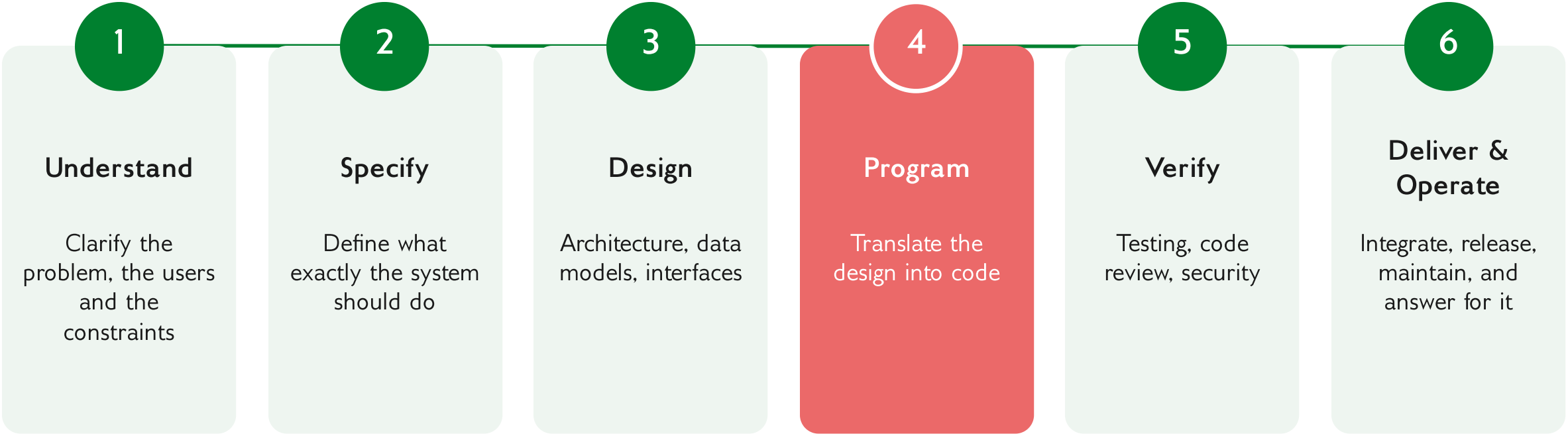
Software became cheaper, so much more software was built. More programmers were needed, not fewer.

3

The work moved “up”

Understanding, specifying, designing, testing: humans state the What, the machine does the How.

What Do Software Engineers Do Today?



Teamwork

Software projects are highly complex socio-technical artefacts

The Decide-Execute-Deliver Sandwich



DECIDE — What to build: requirements, priorities, trade-offs

EXECUTE — Write the code

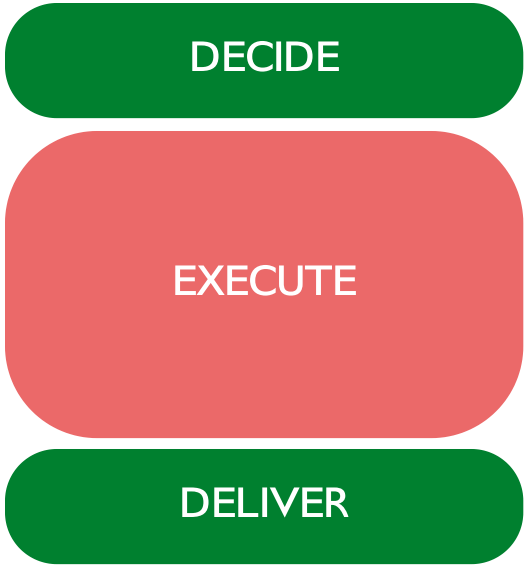
DELIVER — Review, integrate, release, operate, take responsibility

AI compresses the **Execute** layer. **Decide** and **Deliver** resist automation (Narayanan & Kapoor).

Narayanan & Kapoor: Why AI hasn't replaced software engineers, and won't, AI as Normal Technology, 2026. Burger image: AI-generated

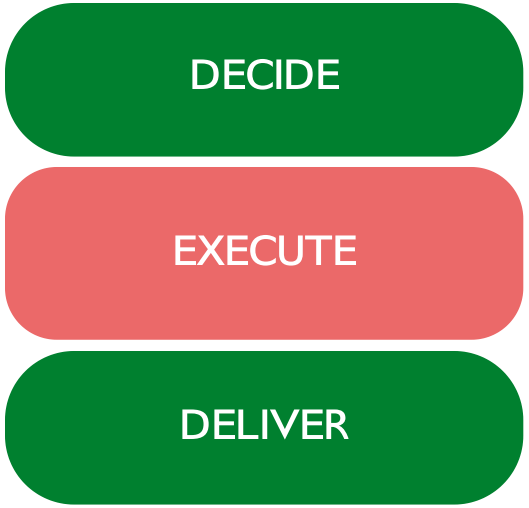
The Compiler Already Squeezed the Sandwich Once

1950s: by hand



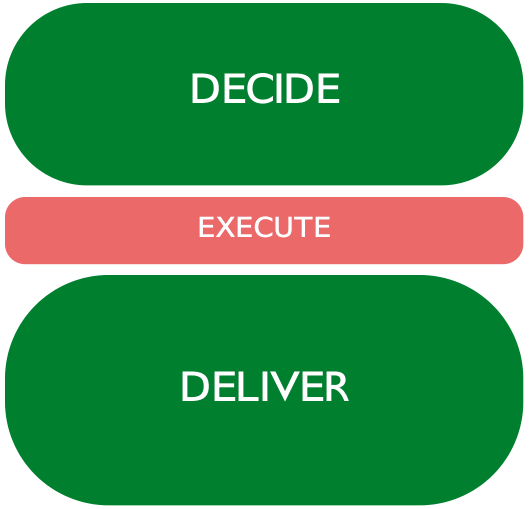
Execute dominates: every line is translated into machine instructions by hand.

From 1957: compilers



Execute shrinks. Software gets cheaper, and much more of it is built.

2026: AI agents



Execute shrinks again. Deciding and delivering become the bottleneck.

Schematic, after Narayanan & Kapoor (2026), AI as Normal Technology

What Remains When Writing Code Gets Cheap?

*The hardest single part of building a software system is **deciding precisely what to build.***

Fred Brooks
No Silver Bullet (1986)

Accidental Complexity

Syntax, memory, boilerplate: how do I tell the machine?

→ Shrinks with every tool: compilers, IDEs, libraries, AI

Essential Complexity

Requirements, trade-offs, special cases: what should the system actually do?

→ Remains, and grows with the size of the systems

Like the Compiler, and Yet Different

Same

What instead of How

Humans describe the goal, the machine generates the code.

Scepticism at first

“Computers cannot write programs”, in 1952 as today.

More demand

Cheaper software → more software, not less work.

Different

Deterministic vs. probabilistic

Nobody reads compiler output, because it is generated deterministically. AI output must be checked, at least indirectly.

Formal vs. natural language

Prompts are ambiguous, like natural-language specifications.

Speed

Three generations of tools since 2021: autocomplete, agents, autonomous agents.

So the work does not disappear. It moves to verifying and delivering.

More Code = More Software? Not So Simple



×25

more lines of code

×1.3

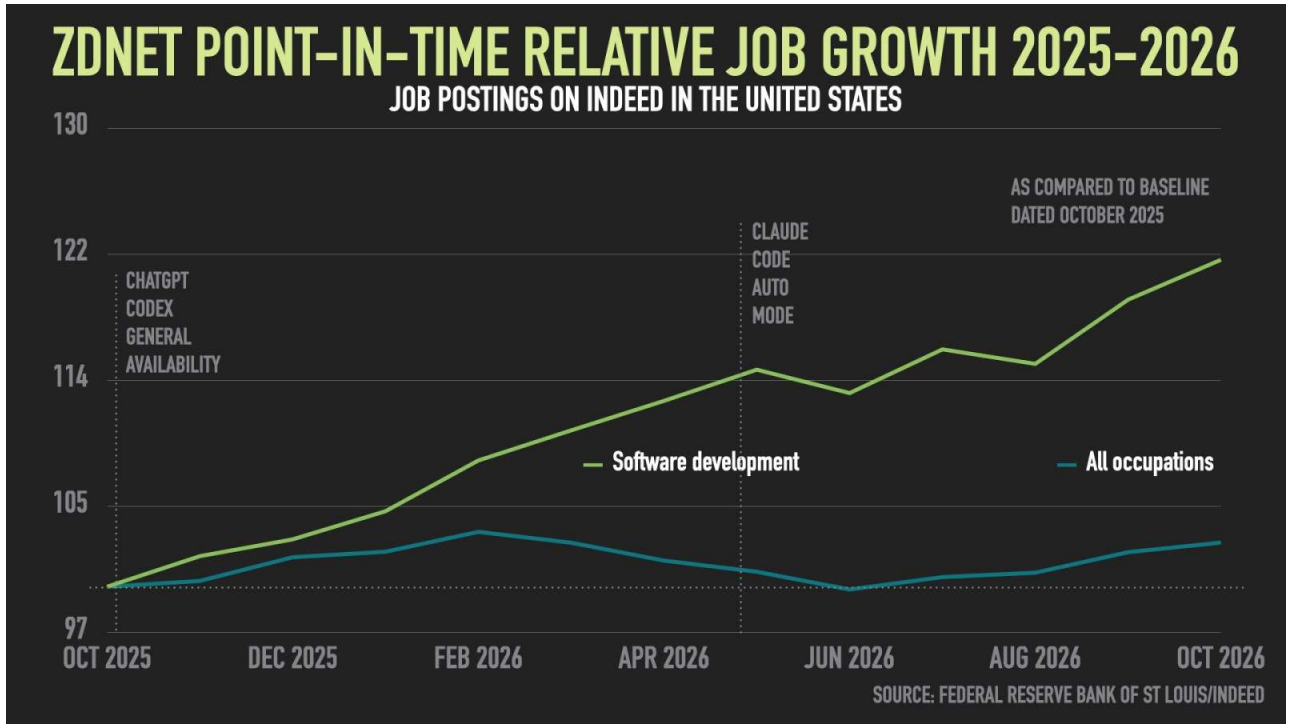
more releases

The closer to delivery, the smaller the effect.

69% of new repositories see no activity after their first month (2021: 60%).

Cumulative effect of three generations of AI tools, log scale. Demirer, Musolff & Yang: Writing Code vs. Shipping Code, NBER Working Paper 35275, 2026, Fig. 1

And the Jobs?

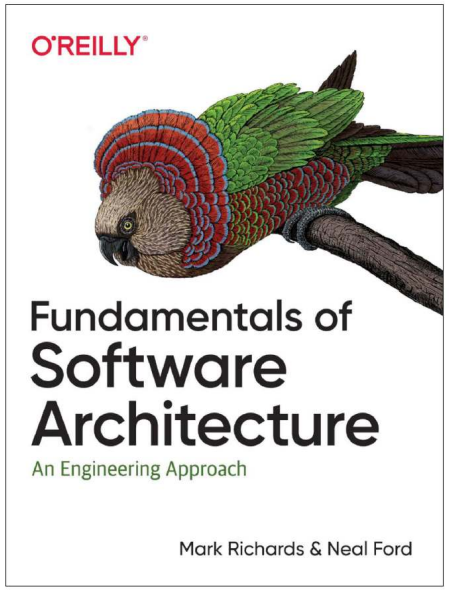


- US job postings on Indeed, Oct 2025 to Oct 2026: **software development about +22%**, all occupations about +3%
- Still about 20% below February 2020, after the hiring peak of 2022
- Gewirtz: companies hire to keep up with the extra projects that AI tools make possible
- As with the compiler: cheaper code, **more demand** for people who decide and deliver

Which layers of the sandwich does **this course** train?

Values read off the charts in D. Gewirtz: The most popular programming languages in 2026, and some good news on developer job growth, ZDNET, 3.11.2026 (chart: David Gewirtz/ZDNET; data: FRED/Indeed)

From Software to Systems



After the focus on **individual components** in the first half of the semester, we now **concentrate on interactions between components**. Specifically, we will discuss the **trade-offs** that are involved in designing these interfaces and interactions.

Many of the concepts that you saw in the course up to now **apply again**!

and systems!

*„Everything in software architecture is a **trade-off**."*

*„**WHY** is more important than **HOW**."*

Software architecture consists of the *structure* of the system, combined with *architecture characteristics* ("-ilities") the system must support, *architecture decisions*, and finally *design principles*.

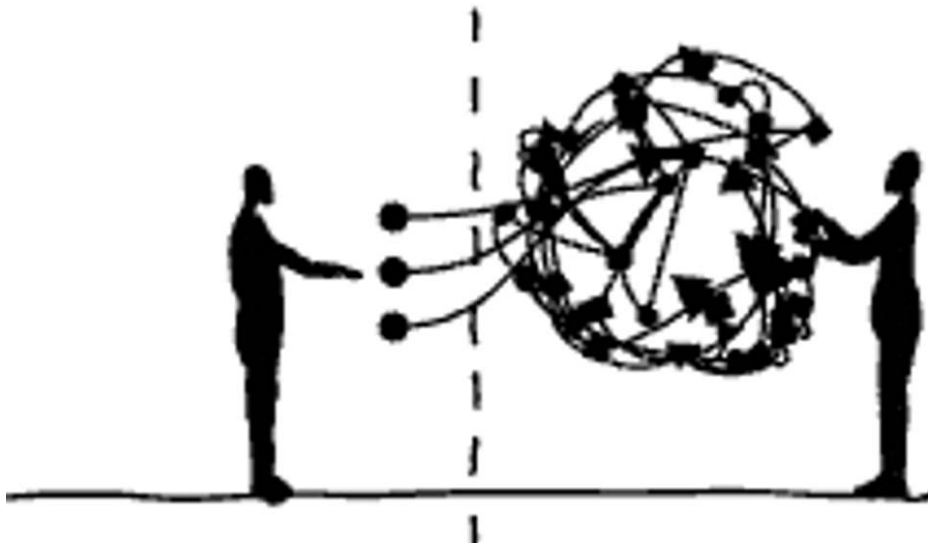
(Selected) Architecture Characteristics

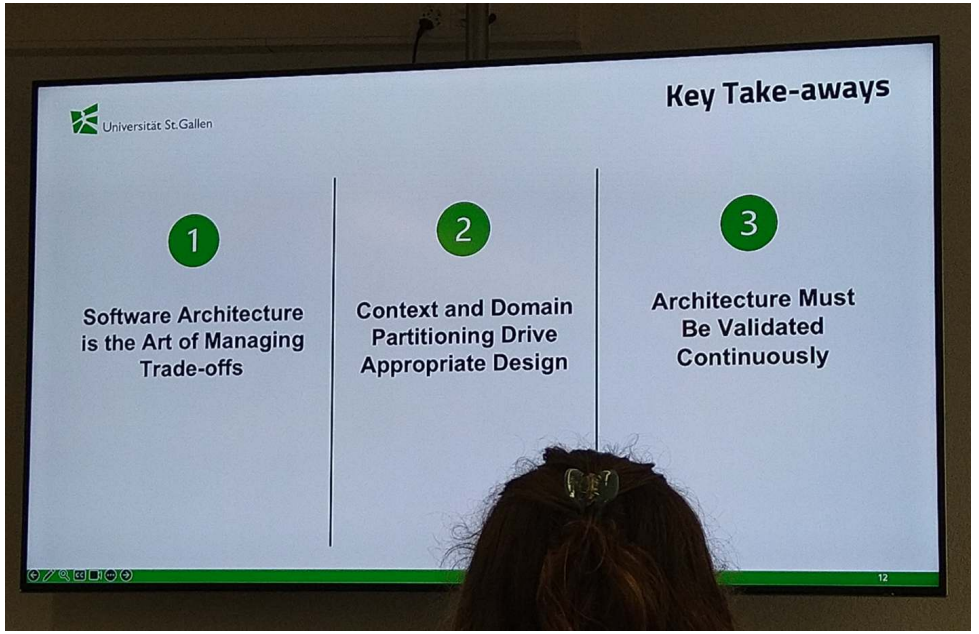
Architecture Characteristic	Definition
Reliability	Refers to the probability of a failure free operation of the system for a specific period
Responsiveness	The ability of a system to complete assigned tasks within a certain time
Availability	Refers to the percentage of time a system is available for its user in a given period
Fault Tolerance	The ability of a system to continue operating in the event of a failure within some of its components
Scalability	The ability to handle large numbers of concurrent users without serious performance degradation
Elasticity	The ability to handle bursts of requests
Data integrity	Refers to the accuracy and consistency (validity) of data over its lifecycle
Data consistency	Refers to whether the same data kept at different places do or do not match
Adaptability	Adaptability refers to the ability to easily change the system according to the requirements.
Extensibility	Refers to the ability to extend a system and the effort needed to implement the extensions
Testability	The degree to which a software artifact supports testing in a given test context
Deployability	The extent to which something is deployable

Pointer Lecture 7b: Principled Architectural Design ↗

From Software to Systems

Overall Goal: Conscious Handling of Architectural Trade-Offs





As the **architect of a (distributed) (information) system**, you should be aware of the **design trade-offs that are induced by specific choices** in order to construct a system **with your desired non-functional properties.**

This is relevant in a (technology) world with projects that **exhibit a broad range of non-functional requirements.**

From Software to Systems

Overall Goal: Multi-Level View on Interoperability



Coupled Bits

Coupled through proprietary interface. **Changing a single bit** might break interaction. Little tolerance to changes on either side.

```
00100010111010010100111010111000100
```



Coupled Schemas

Coupled through open interface and data format. Changing URI handled through uniform interface and HATEOAS. **Changing schema** might break it.

```
GET /temp Accept: application/json
```

Coupled Types

Coupled through open interface and documented media type. **Change in media type** might break it, gracefully.

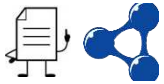
```
GET /temp Accept: application/miele+json
```



Coupled Meanings

Coupled through open interface, documented media type, and agreed-upon standard vocabulary. **Changing meaning of vocabulary terms** might break it.

```
GET /temp Accept: application/ld+json
```



```
/temp a saref:TemperatureSensor
```

```
{"@context": "https://saref.etsi.org/core/Temperature.jsonld",
"Temperature": "20"}
```

Coupled Language

Coupled through natural language that a statistical model interprets at run time. **A different wording or a different model** might change the result.

"How warm is it in the meeting room right now?"



MCP server



AI agents

As the **architect of a (distributed) (information) system**, you should be aware of **different ways** of how to consciously manage the interoperability of system components.

This is relevant in a (technology) world with **increasingly heterogeneous and autonomous components**.

From Software to Systems

Overall Goal: Precision

This doesn't compare **apples to oranges**... it compares **apples to yaks**



Benchmarking performance gRPC vs REST on Node.js

Sergey Onufrienko Jan 23, 2019 · 2 min read

I love REST and JSON for its simplicity. But let's try something new. Let's try gRPC and Protobuf for inter-service communication on Node.js.

Why use gRPC and Protobuf?

1. PRC
2. HTTP/2 for transport

(4) 7 to 10 Times Faster Message Transmission

According to widely-cited tests published by Ruwan Fernando, gRPC API connections are considerably faster than REST API connections. In fact, he reported that they are 7 to 10 times faster:

"gRPC is roughly 7 times faster than REST when receiving data & roughly 10 times faster than REST when sending data for this specific payload. This is mainly due to the tight packing of the Protocol Buffers and the use of HTTP/2 by gRPC."

«Asynchronous», what is that?

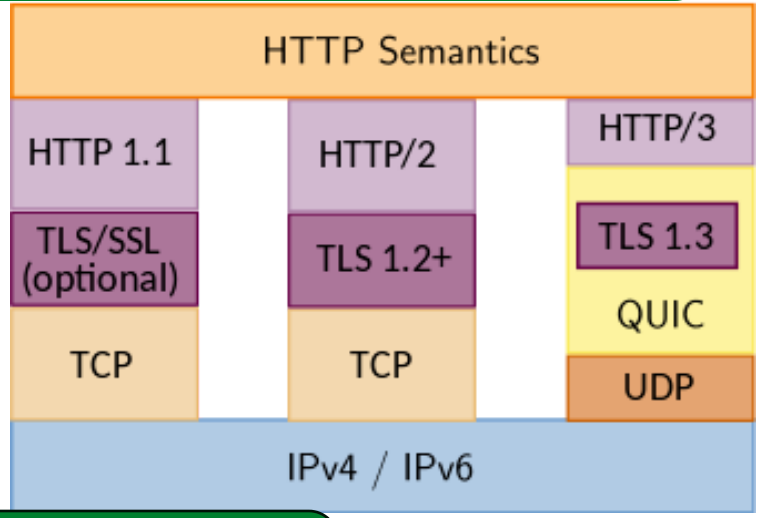
As the **architect of a (distributed) (information) system**, you should use terminology in a clear and well-founded way. Your definitions should be based on accepted expert sources.

This is relevant as our (technology) world needs to be based on **clearly-communicated facts, not on obfuscated beliefs**.

Pointer Lecture 8: Reminder on HTTP/1.1 vs. HTTP/2 ↗
Pointer Lecture 11: The Hypertext Transfer Protocol (HTTP/X) ↗

From Software to Systems

Overall Goal: Precision



Is HTTP a „synchronous“ or an „asynchronous“ protocol?

„blocking“

„client-server“

„request-response“

Does this mean that we can only implement synchronous interactions with HTTP?

But then, aren‘t all protocols that work with TCP „synchronous“ as well?

JS Fetch API

```
fetch(file)
.then(x => x.text())
.then(y => myDisplay(y));
```

AJAX

WebSockets

HTTP/2 Multiplexing

WebHooks

Maybe „synchronous by design“ but „asynchronous in practice“?

Can the system not decide at run time whether it should communicate in a blocking or non-blocking way? ;-)

As the **architect of a (distributed) (information) system**, you should use terminology in a clear and well-founded way. Your definitions should be based on accepted expert sources.

This is relevant as our (technology) world needs to be based on **clearly-communicated facts, not on obfuscated beliefs**.

Lecture Learning Goals

Lecture 7: Understand how to **select fitting architectural constraints** to achieve a desired non-functional profile
Assignment 7: Uniform Interaction? Exploring the REST Architectural Style

Lecture 8: Understand how to **select fitting interaction styles** to achieve a desired non-functional profile
Assignment 8: What Interaction Style? Event-based Interaction using MQTT and W3C WebSub

Lecture 9: Understand **interactive ecosystems** and how data can be separated from applications
Assignment 9: Who Owns the Data? Data Sovereignty with Solid Pods and WebIDs



Plugfest with Pitas (Nov 24, 5 to 8pm)

Lecture 10: Understand that **decoupling may happen on many levels** beyond the immediate API
Assignment 10: Multi-level Decoupling? Decoupling through Semantic Hypermedia

Lecture 11: Understand **what changes** when higher autonomy potential and constrained devices enter the game
Assignment 11: Autonomy and Interactions with Constrained Devices

From Software to Systems

What's with that PlugFest?

Gloriously integrate all groups' implementations

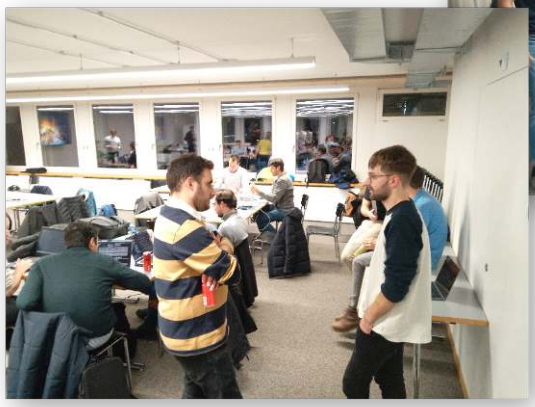
Experience how interoperability testing is done **in practice with your results from** Assignment 7 and Assignment 8

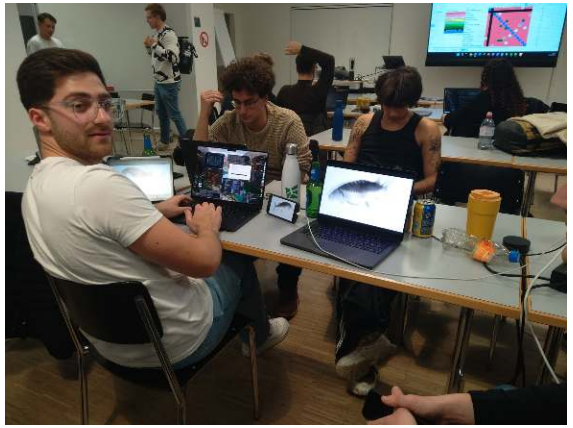
Socialize and have pizza

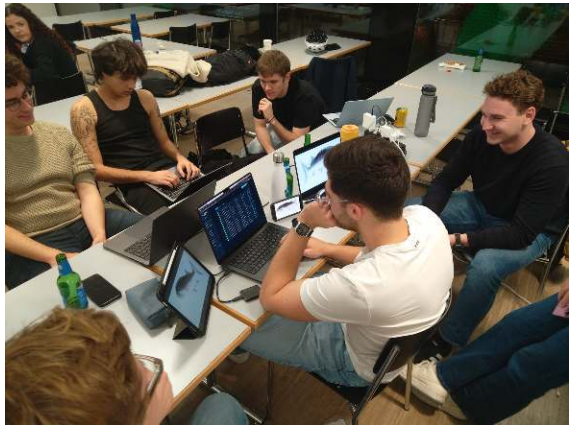










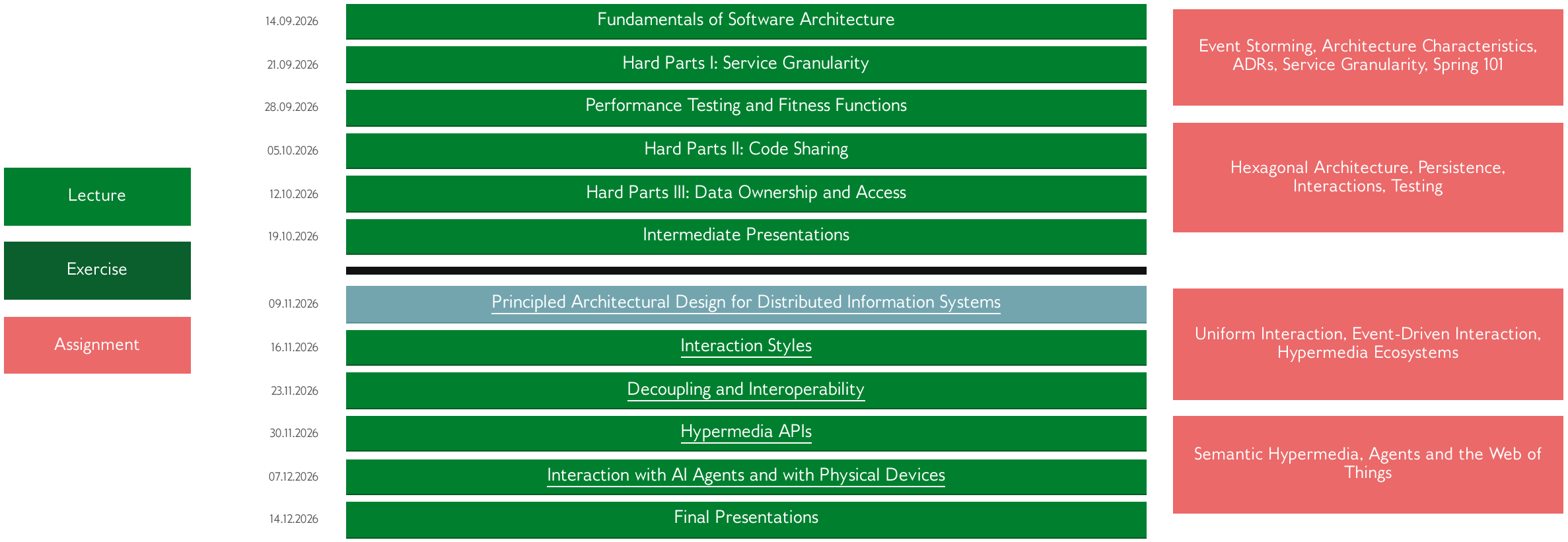






Plugfest with Pitas
(Nov 24, 5 to 8pm)

Syllabus


Lecture
Exercise
Assignment
14.09.2026
Fundamentals of Software Architecture
21.09.2026
Hard Parts I: Service Granularity
28.09.2026
Performance Testing and Fitness Functions
05.10.2026
Hard Parts II: Code Sharing
12.10.2026
Hard Parts III: Data Ownership and Access
19.10.2026
Intermediate Presentations
09.11.2026
Principled Architectural Design for Distributed Information Systems
16.11.2026
Interaction Styles
23.11.2026
Decoupling and Interoperability
30.11.2026
Hypermedia APIs
07.12.2026
Interaction with AI Agents and with Physical Devices
14.12.2026
Final Presentations
Event Storming, Architecture Characteristics, ADRs, Service Granularity, Spring 101
Hexagonal Architecture, Persistence, Interactions, Testing
Uniform Interaction, Event-Driven Interaction, Hypermedia Ecosystems
Semantic Hypermedia, Agents and the Web of Things

Style

I'll be **interactive and illustrative** in the lectures and try to avoid "slide shows"
I'm **illustrating and highlighting** concepts that I find particularly interesting/relevant, not "reading"
You'll need to make sure that you have **read the literature** we point you at
You'll need to make sure that you **ask us** if something isn't clear

Grading Reminder

- Written Exam, Individual Grades (50%)
- Group Project, Individual Grades (50%)
  - Grades given end of semester; improve your projects until final submission
  - Typically the same grade; different grades if relevant divergences

If you need support in your assignment, **don't wait for the next Monday**! Get in touch ;-)

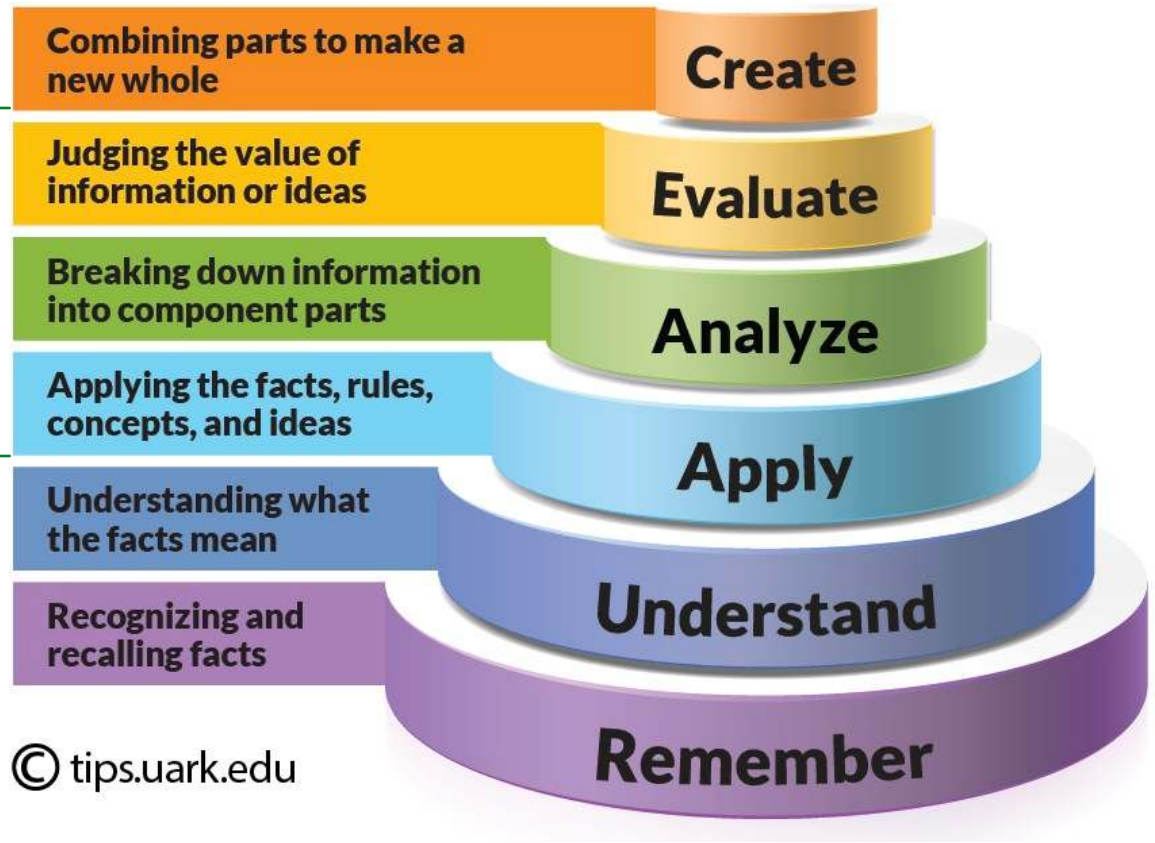
Exam Style

Suggest a system that ...

A colleague suggests this solution to a given (and explained) problem. Do you agree? Why?

In what sense can the performance of gRPC and HTTP be compared?

What is a **drawback of caching**?



Any Questions / Comments / Doubts / Concerns?

Take-Home

- Software architecture means handling trade-offs consciously: every choice favours some non-functional properties over others
- Interoperability can be coupled on several levels: bits, schemas, types, meanings, and language
- Use terms precisely and from accepted sources: whether HTTP is «synchronous» depends on the level you look at
- Assignments 7 to 11 build one system with all groups; the PlugFest on 24.11.2026 tests how well the implementations interoperate

